ENGENHARIA DE SISTEMAS DISTRIBUÍDOS
DOCENTE: Raoni Kulesza
EQUIPE:
Bruno Formiga Brandão - 20210024863
Camila Eduardo Costa De Vasconcelos - 20220054768
Eliane Santos Silva - 20220155152
Gisele Silva Gomes - 20210025824
Klayvert De Andrade Araújo - 20220050259
Lucas Rondineli Lucena Fragoso - 20220005932

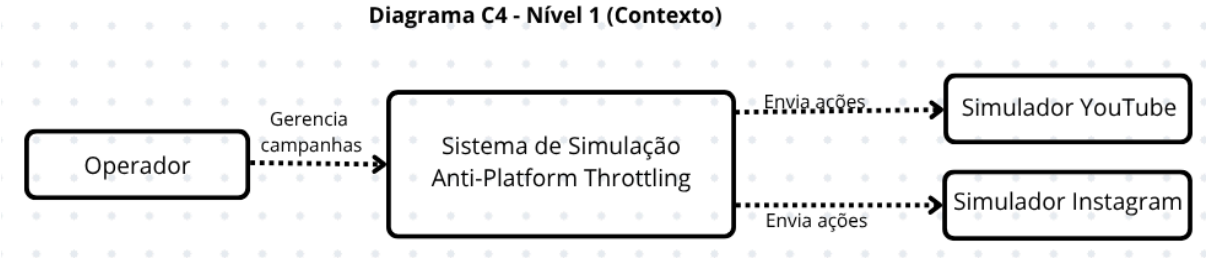
Documentação Inicial do Projeto

Repositório para o projeto:

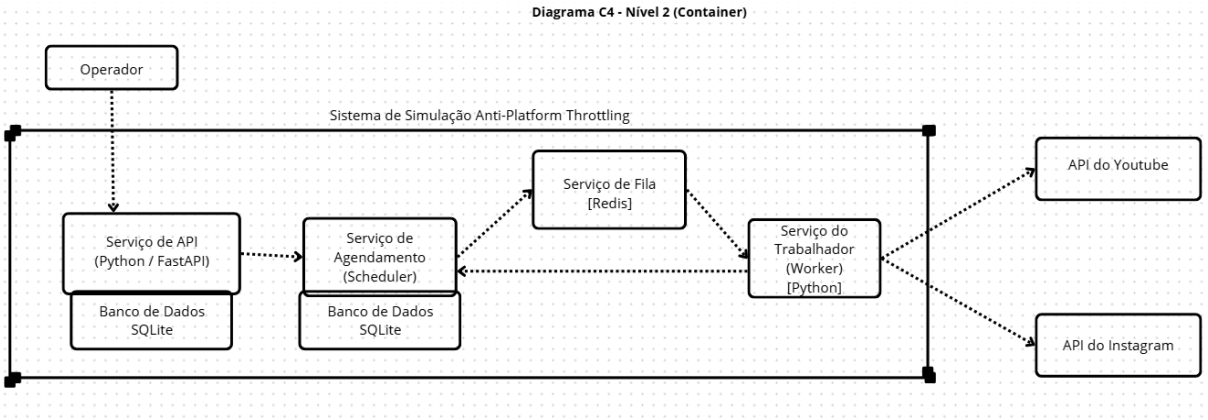
https://github.com/eliane-silva/Projeto-ESD.git

1. Diagrama C4

1.1 Diagrama C4 – Nível 1 (Contexto)



1.2 Diagrama C4 – Nível 2 (Containers)



2. Decisões arquiteturais

2.1 ADR-001: Descoberta e aplicação de thresholds por plataforma

Cada plataforma simulada terá regras próprias de limitação de taxa. O sistema precisa descobrir, usar e atualizar um limite operacional seguro para evitar bloqueios frequentes e, ao mesmo tempo, maximizar throughput.

O sistema adotará uma estratégia de ramp-up progressivo com teto dinâmico por plataforma. A lógica será:

1. iniciar com uma taxa baixa de ações por minuto;
2. aumentar gradualmente a taxa em janelas controladas;
3. ao detectar limitação ou bloqueio, registrar o evento;
4. reduzir o limite operacional para a plataforma;
5. aplicar cooldown com backoff exponencial;
6. retomar com taxa abaixo do último ponto de falha.

Cada plataforma manterá:

- *safe_rate* estimado;
- *last_block_timestamp*;
- *cooldown_until*;
- histórico de falhas e sucessos.

Justificativa

- permite medir empiricamente os thresholds;
- adapta-se a diferentes plataformas;
- reduz necessidade de configuração manual fixa;
- combina segurança operacional com aprendizado experimental.

2.2 ADR-002: Estratégia de distribuição de ações entre plataformas

O sistema precisa distribuir um volume total de ações entre diferentes plataformas, respeitando limites individuais e evitando concentração excessiva em uma só plataforma.

Será adotada uma estratégia de distribuição proporcional baseada na capacidade estimada de cada plataforma, com apoio de filas independentes. A distribuição seguirá os seguintes critérios:

- cada plataforma recebe uma parcela do volume proporcional ao *safe_rate*;
- campanhas ativas entram em uma fila de agendamento;
- o orquestrador recalcula pesos periodicamente;
- caso uma plataforma entre em cooldown, sua carga é redistribuída entre as demais;
- será aplicado *jitter* temporal para evitar padrões rígidos de envio.

Justificativa

- evita sobrecarga concentrada;
- adapta o fluxo ao comportamento observado;
- melhora utilização global da capacidade disponível;
- facilita a comparação entre plataformas.

2.3 ADR-003: Pausa automática e alertas diante de bloqueio

Quando uma plataforma começa a bloquear requisições, continuar insistindo aumenta a instabilidade do experimento e distorce as métricas.

Ao identificar padrão de bloqueio, o sistema ativará pausa automática por plataforma, combinada com alertas e reentrada gradual. Seguindo essas regras iniciais:

- primeiro bloqueio: pausar por 1 unidade de tempo;
- reincidência: 2, 4, 8, 16... até um teto configurável;
- após liberação, retornar com taxa reduzida;
- emitir alerta no painel e registrar evento para análise.

Justificativa

- protege a confiabilidade do experimento;
- evita avalanche de falhas;
- melhora a qualidade dos dados coletados;
- aproxima o sistema de um comportamento resiliente.

3. Definição do stack tecnológico e justificativas

Linguagem: Python
Framework API: FastAPI
Fila: Redis
Banco: SQLite
Containerização: Docker + Docker Compose

O projeto utilizará Python, FastAPI, Redis, SQLite, Docker e Docker Compose, escolhidos por oferecerem simplicidade, integração entre serviços e facilidade de desenvolvimento.

Python foi escolhido por sua sintaxe simples, alta produtividade e amplo ecossistema de bibliotecas. FastAPI foi adotado por facilitar a criação de APIs rápidas, bem organizadas e com documentação automática. Redis será utilizado como fila por ser leve e eficiente na comunicação assíncrona entre scheduler e workers. SQLite foi definido como banco de dados por ser simples de configurar e suficiente para armazenar métricas e resultados do projeto. Docker foi escolhido para padronizar o ambiente de execução e facilitar o uso em diferentes máquinas. Já o Docker Compose permite subir todos os serviços do sistema de forma integrada e centralizada.

4. Padrões arquiteturais que serão implementados

4.1 Feature Flag

Área: Segurança

O padrão Feature Flag será utilizado para habilitar ou desabilitar funcionalidades do sistema de forma controlada, sem necessidade de alterar a arquitetura principal nem remover código já implementado. No nosso projeto, esse padrão será importante porque o ambiente é experimental e várias estratégias de

controle de engajamento poderão ser ativadas gradualmente, comparadas entre si e desativadas rapidamente caso provoquem comportamento inadequado.

No nosso projeto, as feature flags serão aplicadas principalmente sobre funcionalidades sensíveis do orquestrador, como:

- modo de descoberta automática de thresholds por plataforma;
- ativação da pausa automática após bloqueios;
- uso de jitter na distribuição temporal das ações;
- redistribuição dinâmica de carga entre plataformas simuladas.

4.2 Rate Limit / Throttling Pattern

Área: Desempenho

O padrão Rate Limit / Throttling será um dos elementos centrais da arquitetura, pois o objetivo do projeto é justamente simular e medir estratégias para evitar excesso de interações em plataformas externas. Esse padrão consiste em controlar explicitamente a taxa de envio de requisições ou ações para que o sistema não ultrapasse limites aceitáveis de processamento ou detecção.

No nosso projeto, ele será aplicado no orquestrador de campanhas, que controlará a quantidade de ações por minuto enviadas para cada plataforma simulada. Esse limite não será necessariamente fixo, ele poderá variar conforme o threshold estimado de cada plataforma, permitindo que o sistema se adapte ao comportamento observado durante a execução. Assim, se uma plataforma simulada tolerar menos interações por minuto, o orquestrador reduzirá automaticamente o ritmo de envio. Caso tolere mais, o sistema poderá ampliar a vazão dentro de uma faixa segura.

4.3 Queues / PubSub / Messaging Fanout

Área: Escalabilidade

O padrão Queues / PubSub / Messaging Fanout será utilizado para desacoplar a geração de ações da execução efetiva dessas ações nas plataformas simuladas. Em vez de o sistema enviar todas as interações diretamente no momento em que uma campanha é criada, as ações serão primeiro publicadas em filas, para depois serem consumidas pelos workers responsáveis por cada plataforma.

No nosso projeto, esse padrão será aplicado para organizar o fluxo entre a entrada de campanhas e a execução das interações. O orquestrador receberá as campanhas e transformará os objetivos de engajamento em tarefas menores, que serão distribuídas por meio de filas. A arquitetura poderá trabalhar com:

- uma fila por plataforma simulada;
- uma fila por campanha;
- ou uma fila global com roteamento interno para os workers corretos.

5. Setup inicial do ambiente

Inicialmente, foi decidido utilizar a seguinte estrutura para o ambiente do projeto:

```
project/
|-- docker-compose.yml
|-- services/
|   |-- fake_youtube/
|   |   |-- app.py
|   |   +-- Dockerfile
|   |-- fake_instagram/
|   |   |-- app.py
|   |   +-- Dockerfile
|   |-- worker/
|   |   |-- worker.py
|   |   +-- Dockerfile
|   |-- scheduler/
|   |   |-- scheduler.py
|   |   +-- Dockerfile
```

Conforme representado no diagrama C4, optou-se pela criação de duas plataformas simuladas (YouTube e Instagram), com o objetivo de permitir a execução de interações automatizadas em um ambiente controlado, evitando violações de termos de uso de plataformas reais.

A arquitetura proposta é composta por um serviço de API, responsável pela geração de tarefas, um *scheduler*, responsável pela distribuição das tarefas ao longo do tempo (incluindo estratégias como *jitter*), e por múltiplos *workers*, responsáveis por consumir essas tarefas e executar as interações nas APIs simuladas.

Além disso, foi incluído um mecanismo de persistência simples para armazenamento de métricas, permitindo registrar informações como taxa de sucesso, erros e ocorrências de *throttling*. Esses dados são fundamentais para a análise do comportamento do sistema e validação das estratégias de controle de carga implementadas.

6. Divisão de responsabilidades entre os integrantes

A divisão de responsabilidades foi organizada de modo que cada integrante ficasse responsável por um serviço ou módulo principal do sistema. Um integrante ficou encarregado do Scheduler, incluindo a lógica de agendamento e distribuição de tarefas, outro das APIs simuladas do YouTube e Instagram, outro da API principal e da interface do operador, outro do monitoramento e persistência em

banco de dados, outro dos Workers responsáveis pela execução assíncrona, e o último dos testes, integração final e validação da solução.

API Gateway + filas - Gisele Silva Gomes
Fake YouTube + Fake Instagram - Eliane Santos Silva
API do Scheduler + front - Bruno Formiga Brandão
Monitoramento + banco - Camila Eduardo Costa de Vasconcelos
Workers - Lucas Rondineli Lucena Fragoso
Testes + integração final - Klayvert De Andrade Araújo